TVM: Automated E2E Optimizing Compiler for Deep Learning

Presented by Group 6
Christopher Lin | Jakob Therkelsen | Jayanaka Dantanarayana | Johnson He | Jona Grodecki | Kellen Kanarios

Motivating Example: cuDNN

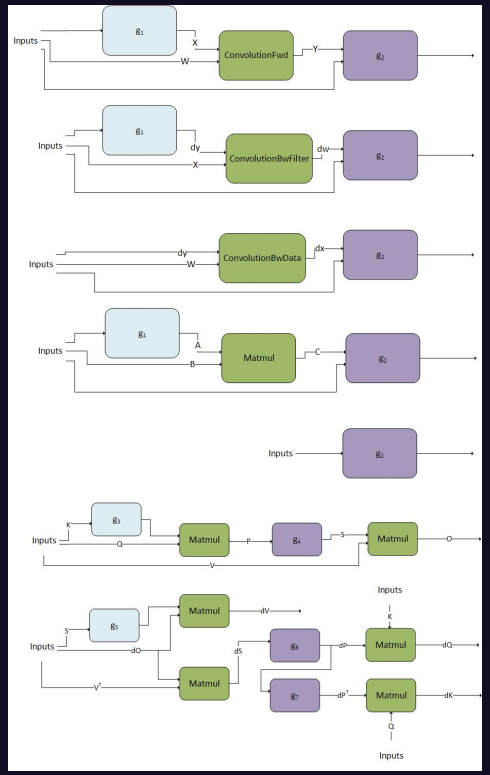
1. `ConvolutionFwd` fusions
   $g_2(\mathrm{Y} = \text{convolutionFwd}\ (\mathrm{X} = g_1\ (\text{inputs}), \mathrm{W}), \text{inputs})$
2. `ConvolutionBwFilter` fusions
   $g_2(\mathrm{dw} = \text{convolutionBwFilter}\ (\mathrm{dy} = g_1\ (\text{inputs}), \mathrm{X}), \text{inputs})$
3. `ConvolutionBwData` fusions
   $g_2(\mathrm{dx} = \text{convolutionBwData}\ (\mathrm{dy}, \mathrm{W}), \text{inputs})$
4. `MatMul` fusions
   $g_2(\mathrm{C} = \text{matmul}\ (\mathrm{A} = g_1\ (\text{inputs}), \mathrm{B}), \text{inputs})$
5. Pointwise fusions
   $g_2(\text{inputs})$
6. `Mha-Fprop` fusions
   $\mathrm{O} = \text{matmul}\ (\mathrm{S} = g_4\ (\mathrm{P} = \text{matmul}\ (\mathrm{Q}, g_2\ (\mathrm{K})), \mathrm{V}))$
7. `Mha-Bprop` fusions
   This pattern is executed in a fused pattern in a single kernel.

   $\mathrm{dV} = \text{matmul}\ (g_5\ (\mathrm{S}), \mathrm{dO})$

   $\mathrm{dS} = \text{matmul}\ (\mathrm{dO}, V^T)$

   $\mathrm{dQ} = \text{matmul}\ (g_6\ (\mathrm{dS}), \mathrm{K})$

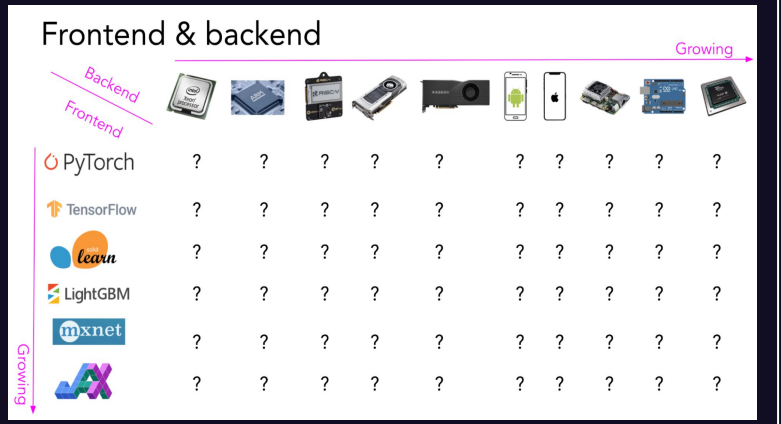
   $\mathrm{dK} = \text{matmul}\ (\mathrm{Q}, g_7\ (\mathrm{dS}))$

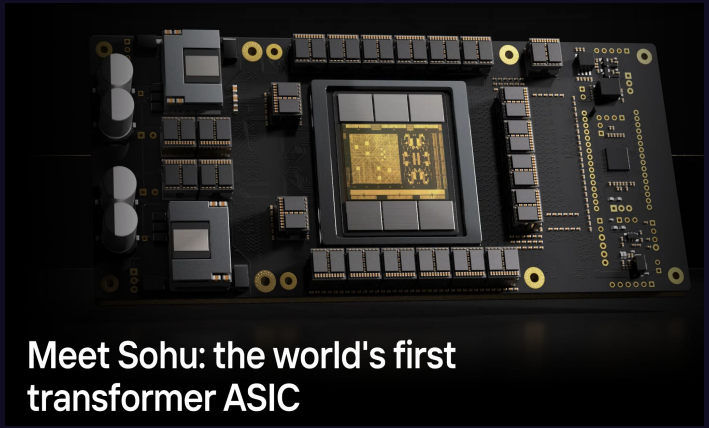


	Limitations to g_1
`ConvolutionFwd` fusions	• Fusion operations on input tensors can be only a chain of three specific pointwise operations, in this exact order: `Pointwise:mul`, `Pointwise:add`, and `Pointwise:ReLU`. This specific support is added to realize convolution batch norm fusion use cases. • All tensors involved can only be FP16. • `Pointwise:mul` can only be with a tensor of scalars per channel. • `Pointwise:add` can only be a column broadcast.
`ConvolutionBwFilter` fusions	Same limitations specified for `ConvolutionFwd` fusions apply here.
`ConvolutionBwData` fusions	No fusion on input tensors for backward data convolution is supported.
`MatMul` fusions	• Can be any combination of pointwise operations. • Only fusible with operand A, not with B. • Operand A should have an FP16 data type. • Broadcasted input can have any data type. • Compute type is FP32 only.
Pointwise fusions	Not Applicable

Pattern	Layout Requirement
`ConvolutionFwd`, `ConvolutionBwFilter`, `ConvolutionBwData` fusions	• All tensors are fully packed NHWC.
`MatMul` fusions	• Input operands can have either row-major or all column-major. • In g_1, the tensor operating with Matrix A (dim[B, M, K]) can be either a scalar with dim[1, 1, 1], a row vector with dim[B, M, 1], a column vector with dim[B, 1, K], or a full matrix with dim[B, M, K]. • In g_2, all I/O tensors should be either all row-major or all column-major.
Pointwise fusions	• If all tensors are 3D, the same layout requirements as matmul g_2. • If all tensors are 4D or 5D, the same requirements as `ConvolutionFwd`, `ConvolutionBwFilter`, `ConvolutionBwData` layout.
`Mha-Fprop` fusions	• All I/O tensors need to have 4 dimensions, with the first two denoting the batch dimensions. The usage of rank-4 tensors in matmul ops can be read from the NVIDIA cuDNN Backend API documentation. • The contracting dimension (dimension K) for the first matmul needs to be 64. • The non contracting dimension (dimensions M and N) for the first matmul needs to be equal and less than or equal to 512. • The last dimension (corresponding to hidden dimensions) in Q, V and O is expected to have stride 1. • For the K tensor, the stride is expected to be 1 for the 2nd last dimension. • The S tensor is expected to have the `CUDNN_ATTR_TENSOR_REORDERING_MODE` set to `CUDNN_TENSOR_REORDERING_F16x16`
`Mha-Bprop` fusions	• All I/O tensors need to have 4 dimensions, with the first two denoting the batch dimensions. The usage of rank-4 tensors in matmul ops can be read from the NVIDIA cuDNN Backend API documentation. • The contracting dimension (dimension K) for the second matmul needs to be 64. • The contracting dimension (dimension K) for the 1st, 2nd and 3rd matmul needs to be equal and less than or equal to 512. • The last dimension (corresponding to hidden dimensions) in Q, K, V, O and dO is expected to have stride 1. • The S tensor and dP tensor is expected to have the `CUDNN_ATTR_TENSOR_REORDERING_MODE` set to `CUDNN_TENSOR_REORDERING_F16x16`

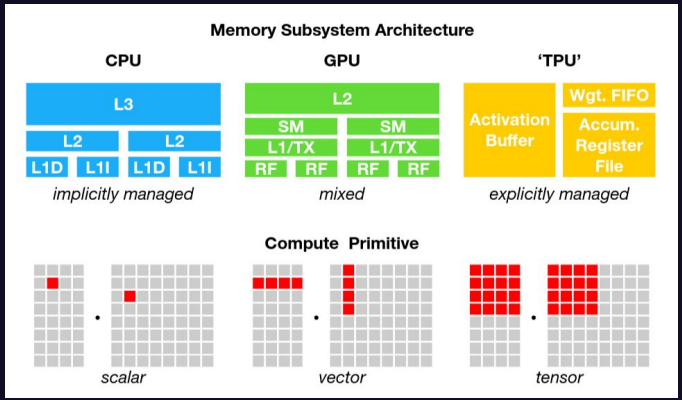
cuDNN on an iPhone?

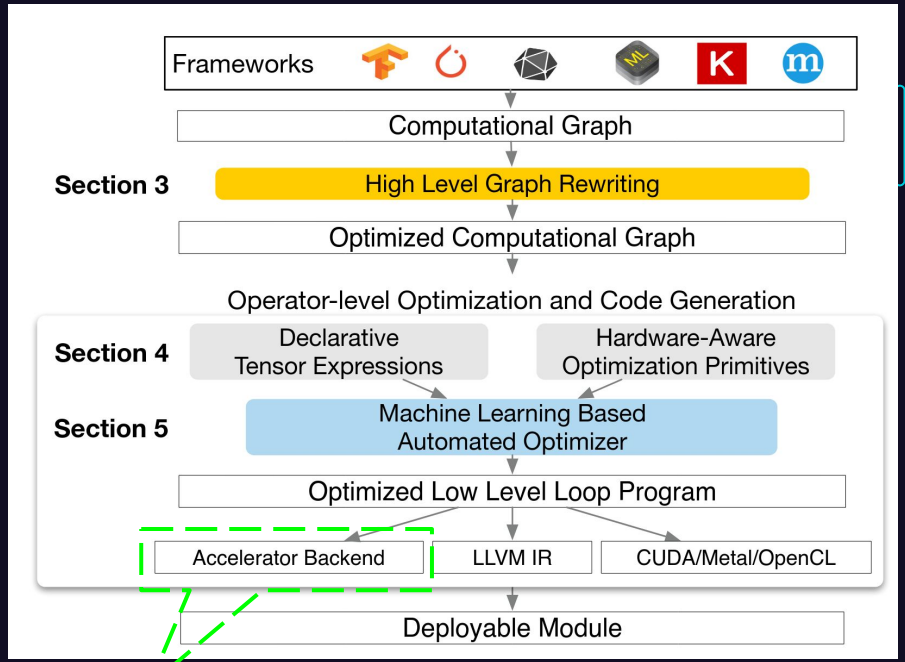




Deep Learning

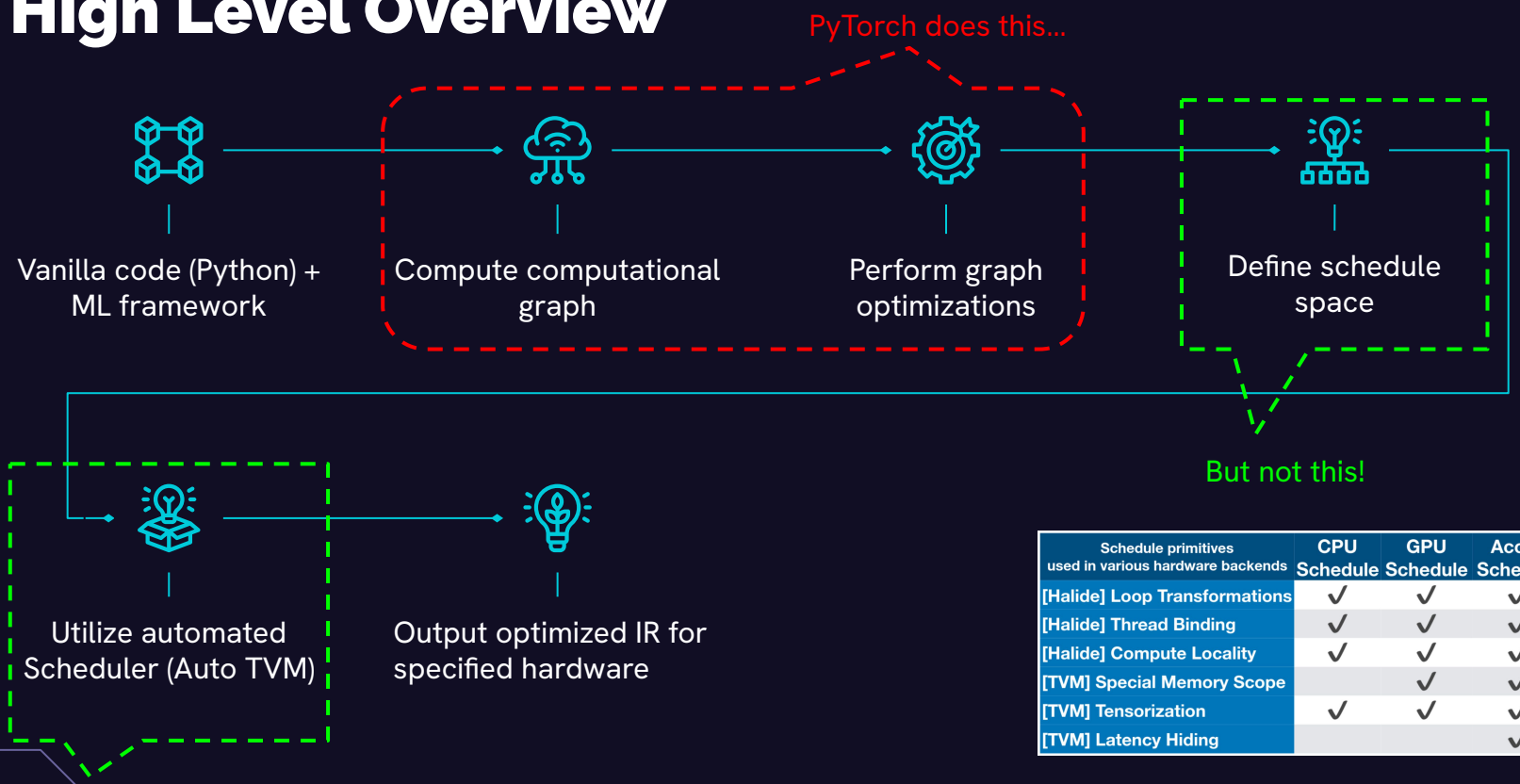
```
@dynamo.optimize(tvm_compiler)
class MyModel(torch.nn.Module):

# YOUR EXISTING MODEL HERE
```





Easily extend to your hardware of choice!

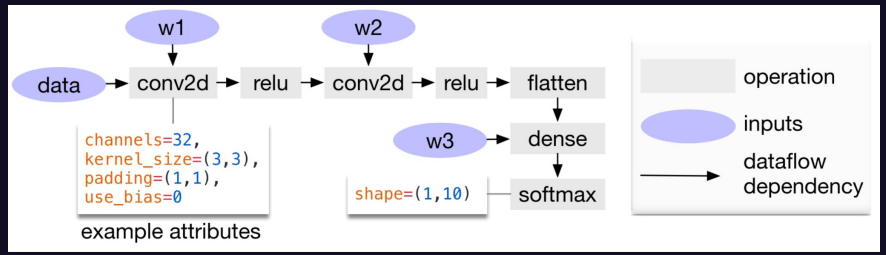
High Level Overview



Schedule primitives used in various hardware backends	CPU Schedule	GPU Schedule	Accel. Schedule
[Halide] Loop Transformations	✓	✓	✓
[Halide] Thread Binding	✓	✓	✓
[Halide] Compute Locality	✓	✓	✓
[TVM] Special Memory Scope		✓	✓
[TVM] Tensorization	✓	✓	✓
[TVM] Latency Hiding			✓

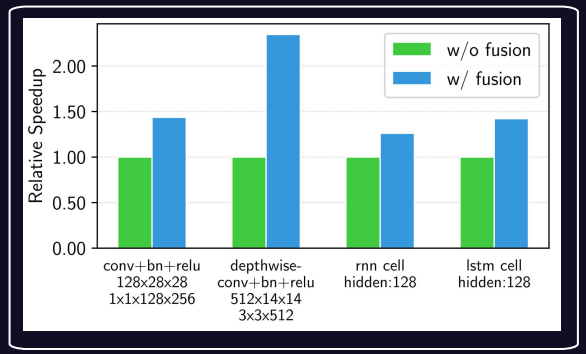
Computation Graphs

- Transform Comp. Graphs into functionally equivalent graphs for optimization.
- Think back to how optimizations were done to LLVM IR.
  - Operator Fusion
  - Constant-folding
  - Static Memory Planning
  - Data Layout Transformations

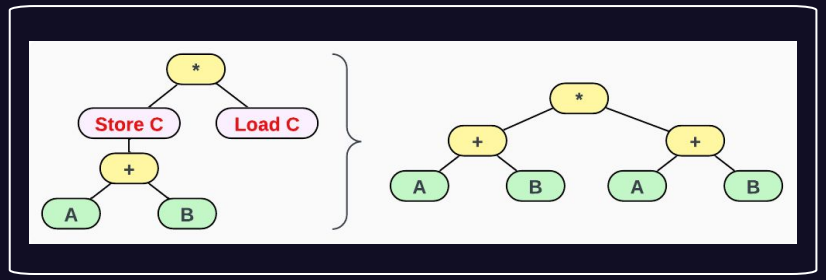


Computation Graphs: Operator Fusion

- Avoid storing intermediate results
  - Reduce memory bandwidth usage
- Particularly useful for GPU optimization
- **Speedup 1.2x-2x**



- Rule-based :
  - ***Injective Operator Fusion :***
    - Can fuse simple, element-wise operations (e.g., add, mul, max, etc.)
  - ***Reduction Operator Fusion :***
    - Supports the fusion of reduction operators (e.g., sum, min, max)
  - ***Complex Operator Fusion :***
    - Capable of fusing complex operators (e.g., conv2d, matmul, batchnorm)



Operator Fusion: A Problem !

- Fine grained optimization is possible under one assumption.

  For all fused operator combinations, kernel support already exists

  which is not the case!

- Eg. cuDNN only supports mul → add → ReLU in that order
  - The fused operations are predefined and optimized by cuDNN
  - Hence, limited flexibility in customizing or reordering them outside these built-in patterns.
- Need implementations for arbitrary operator combinations which is not feasible.
  - Instead generate operators themselves!

Generation: Tensor Expression Language

- Unified representation of program logic.
- No loop structure or execution details
  - Decoupled compute/schedule principle
- Operator is generated ensuring logical correctness of compute

Code sample

```
m, n, h = t.var('m'), t.var('n'), t.var('h')
A = t.placeholder((m, h), name='A')
B = t.placeholder((n, h), name='B')
k = t.reduce_axis((0, h), name='k')

C = t.compute((m, n), lambda y, x:
                t.sum(A[k, y] * B[k, x], axis=k))
```

Compute rule

shape

Generation: Overview

Tensor Expression

Select Schedule Primitive

Final Schedule

Code Lowering

Low Level Code

```
A = t.placeholder((1024, 1024))
B = t.placeholder((1024, 1024))
k = t.reduce_axis((0, 1024))
C = t.compute((1024, 1024), lambda y, x:
                t.sum(A[k, y] * B[k, x], axis=k))
s = t.create_schedule(C.op)
```

```
for y in range(1024):
  for x in range(1024):
    C[y][x] = 0
    for k in range(1024):
      C[y][x] += A[k][y] * B[k][x]
```

+ Loop Tiling

```
yo, xo, ko, yi, xi, ki = s[C].tile(y, x, k, 8, 8, 8)
```

```
for yo in range(128):
  for xo in range(128):
    C[yo*8:yo*8+8][xo*8:xo*8+8] = 0
    for ko in range(128):
      for yi in range(8):
        for xi in range(8):
          for ki in range(8):
            C[yo*8+yi][xo*8+xi] +=
              A[ko*8+ki][yo*8+yi] * B[ko*8+ki][xo*8+xi]
```

+ Cache Data on Accelerator Special Buffer

```
CL = s.cache_write(C, vdla.acc_buffer)
AL = s.cache_read(A, vdla.inp_buffer)
# additional schedule steps omitted …
```

+ Map to Accelerator Tensor Instructions

```
s[CL].tensorize(yi, vdla.gemm8x8)
```

```
inp_buffer AL[8][8], BL[8][8]
acc_buffer CL[8][8]
for yo in range(128):
  for xo in range(128):
    vdla.fill_zero(CL)
    for ko in range(128):
      vdla.dma_copy2d(AL, A[ko*8:ko*8+8][yo*8:yo*8+8])
      vdla.dma_copy2d(BL, B[ko*8:ko*8+8][xo*8:xo*8+8])
      vdla.fused_gemm8x8_add(CL, AL, BL)
    vdla.dma_copy2d(C[yo*8:yo*8+8,xo*8:xo*8+8], CL)
```

schedule → schedule transformation ⇢ corresponding low-level code

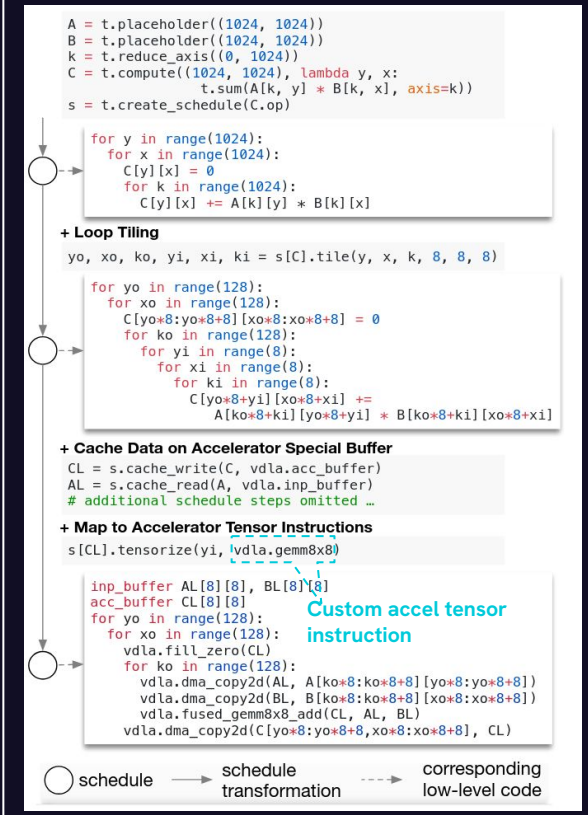
Optimization: Tensorization

- Extend schedule primitive search to custom hardware

```
w, x = t.placeholder((8, 8)), t.placeholder((8, 8))
k = t.reduce_axis((0, 8))
y = t.compute((8, 8), lambda i, j:
                t.sum(w[i, k] * x[j, k], axis=k))

def gemm_intrin_lower(inputs, outputs):
   ww_ptr = inputs[0].access_ptr("r")
   xx_ptr = inputs[1].access_ptr("r")
   zz_ptr = outputs[0].access_ptr("w")
   compute = t.hardware_intrin("gemm8x8", ww_ptr, xx_ptr, zz_ptr)
   reset = t.hardware_intrin("fill_zero", zz_ptr)
   update = t.hardware_intrin("fuse_gemm8x8_add", ww_ptr, xx_ptr, zz_ptr)
   return compute, reset, update

gemm8x8 = t.decl_tensor_intrin(y.op, gemm_intrin_lower)
```

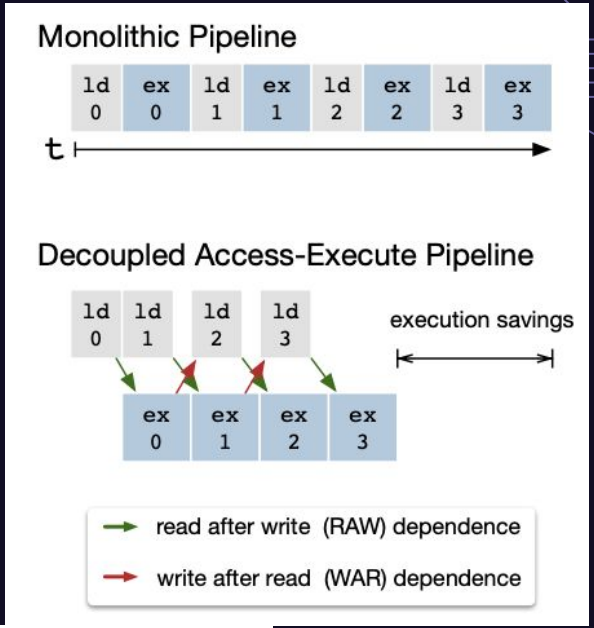
declare behavior

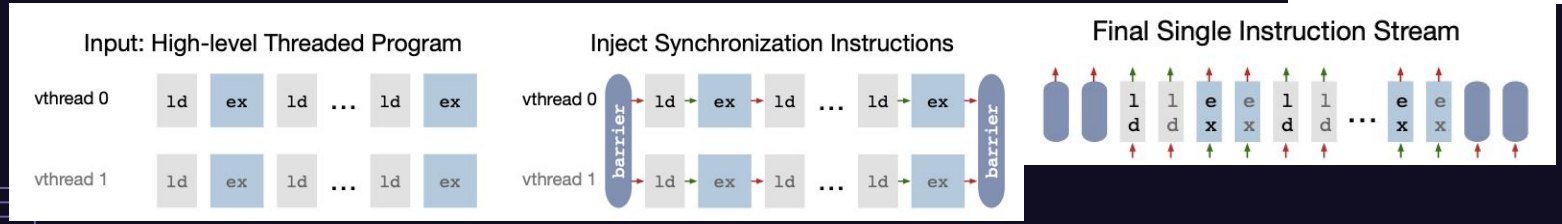
lowering rule to generate hardware intrinsics to carry out the computation



Optimization: Memory Latency Hiding

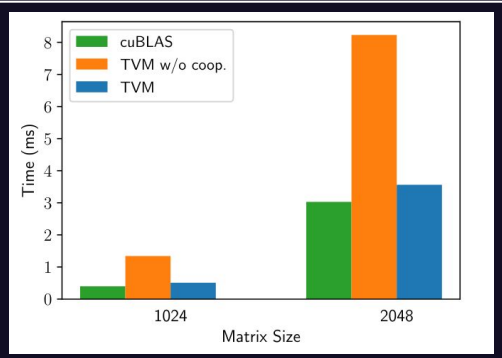
- CPU uses simultaneous multithreading or hardware prefetching
- GPU has warps of threads it can swap in/out
- Targeting DAE pipelines on specialized compute
  - User specifies parallel “virtual threads” similar to multithreading
  - TVM inserts synchronization primitives to ensure program correctness
  - Instructions interleaved in single instruction stream to allow for parallel memory/compute operations





Optimization: Memory Scope

- Schedule space can mark AS and BS as shared.
- Now can collaboratively load memory between threads.



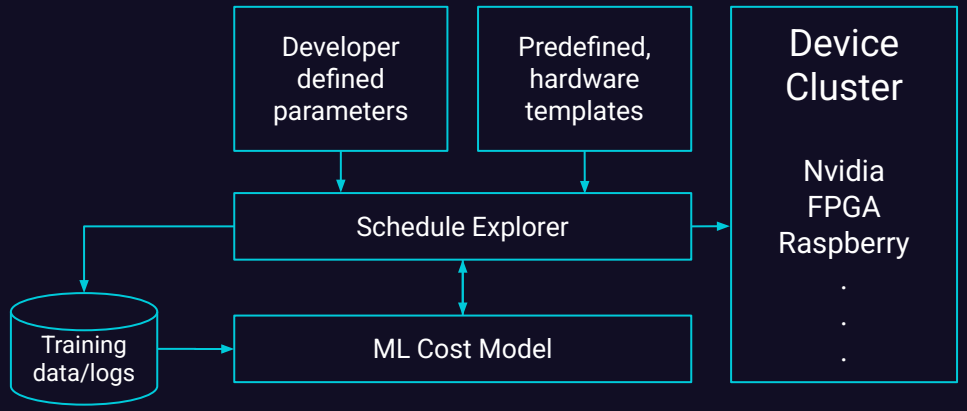

```
for thread_group (by, bx) in cross(64, 64):
  for thread_item (ty, tx) in cross(2, 2):
    local CL[8][8] = 0
    shared AS[2][8], BS[2][8]
    for k in range(1024):
      for i in range(4):
        AS[ty][i*4+tx] = A[k][by*64+ty*8+i*4+tx]
      for each i in 0..4:
        BS[ty][i*4+tx] = B[k][bx*64+ty*8+i*4+tx]
      memory_barrier_among_threads()
      for yi in range(8):
        for xi in range(8):
          CL[yi][xi] += AS[yi] * BS[xi]
      for yi in range(8):
        for xi in range(8):
          C[yo*8+yi][xo*8+xi] = CL[yi][xi]
```

All threads cooperatively load AS and BS in different parallel patterns

Barrier inserted automatically by compiler

Generation: ML-Cost based optimization

- domain-specific knowledge (knob)
  - Knob example (hyperparameter tuning) for 2d-mul:
    cfg.define_knob("tile_y", [1, 2, 4, 8, 16])
  - Others: loop unrolling, thread binding, ||'ization, etc.
- Hardware templates inform valid optimizations and costs
- Schedule explorer "steps" in nearby search space to lower cost

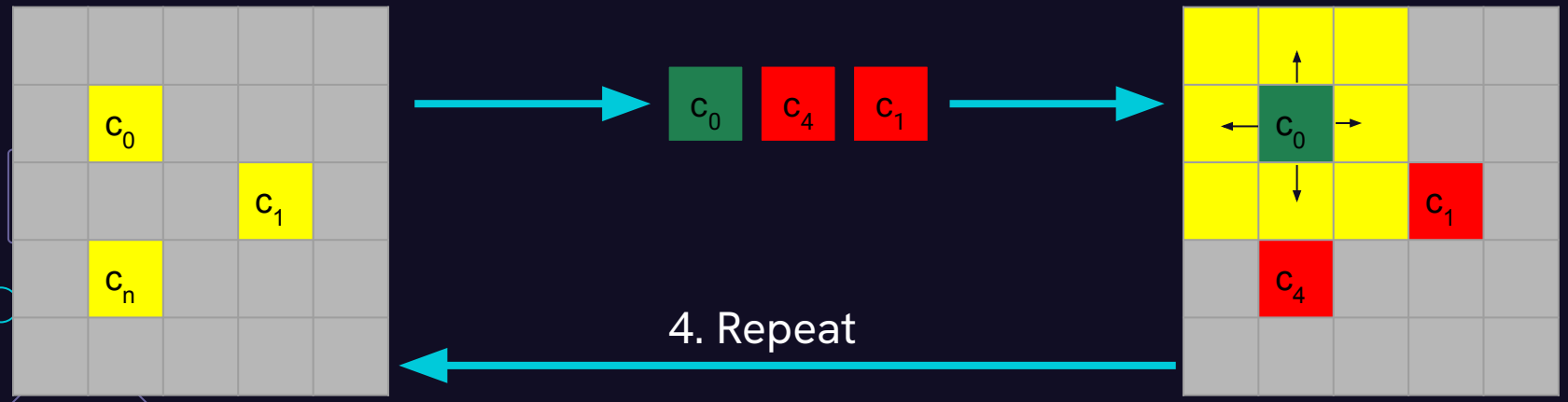


Generation: The Schedule Explorer

1. Initialize Randomly or from previous training

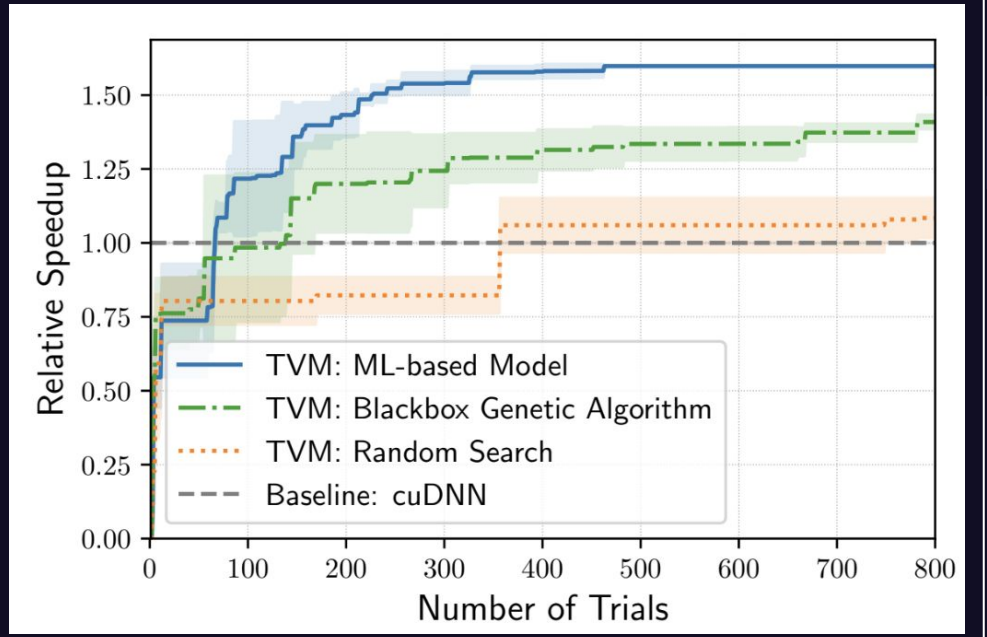
2. Measure using cost($c_{i \to n}$), rank the minibatch

3. Run top-k configs, record actual performance and "walk" to nearby configs.

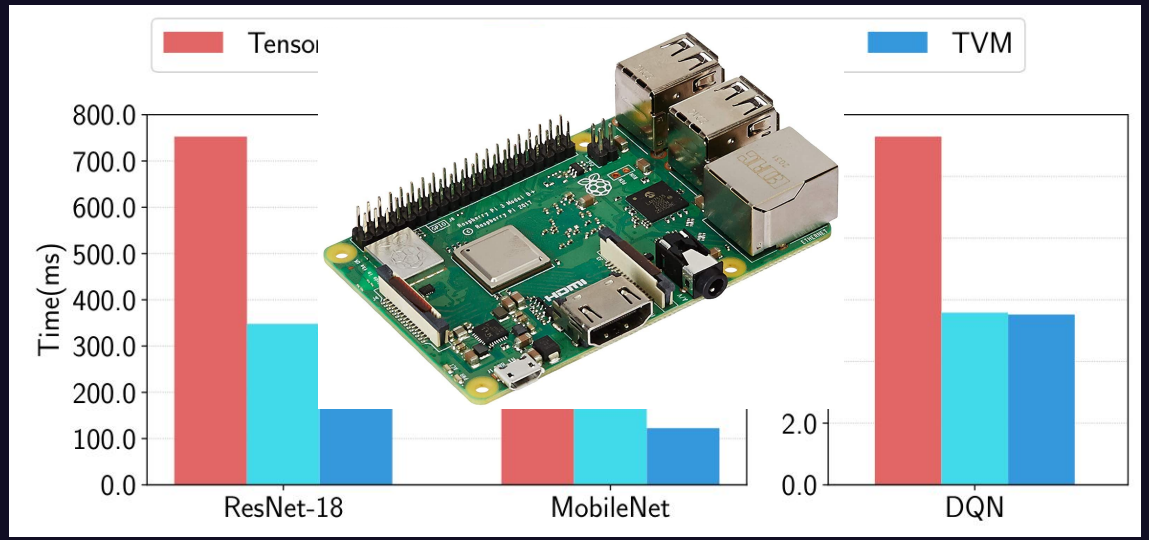


Design Considerations

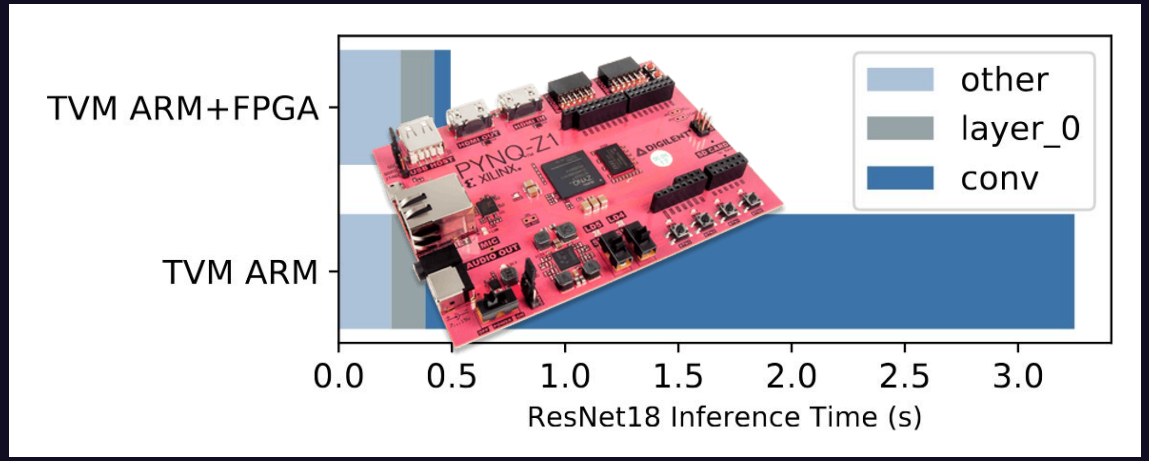
- Model is trained from exploration (actual profile data)
- Quality vs. speed
- What ML model?
  - Supervised Learning
  - Neural nets
  - etc.



Results


Tenso
TVM
Time(ms)
800.0
700.0
600.0
500.0
400.0
300.0
200.0
100.0
0.0
2.0
0.0
ResNet-18
MobileNet
DQN

Results


TVM ARM+FPGA
TVM ARM
other
layer_0
conv
0.0
0.5
1.0
1.5
2.0
2.5
3.0
ResNet18 Inference Time (s)

Limitations/Weaknesses

- Performance Trade-Offs on Some Hardware:
  - TVM performs best on general hardware
  - Highly specialized hardware with manual optimizations will always beat TVMs
- Challenges in Debugging and Profiling
  - Auto scheduler and optimization techniques can obscure the underlying code
- Optimized only for single device efficiency (making multi-GPU training and inference difficult)
  - Lack of built-in distributed operator primitives
  - Lack of support for inter-device communication/synchronization
- AutoTVM's optimizations require several test examples on that hardware

Commentary

- It’s, well, LLVM for deep learning!

- ~~All the world’s a VAX~~

Fin

Appendix Slide: Feature Extraction

Schedule Explorer

Query: Loop AST

```
for yo in range(4):
  for xo in range(4):
    C[yo*2:yo*2+2][xo*2:xo*2+2] = 0
    for ko in range(8):
     for yi in range(2):
        for xi in range(2):
          C[yo*2+yi][xo*2+xi] +=
            A[k][yo*2+yi] * B[k][xo*2+xi]
```

alternatively, we can feed AST to TreeRNN

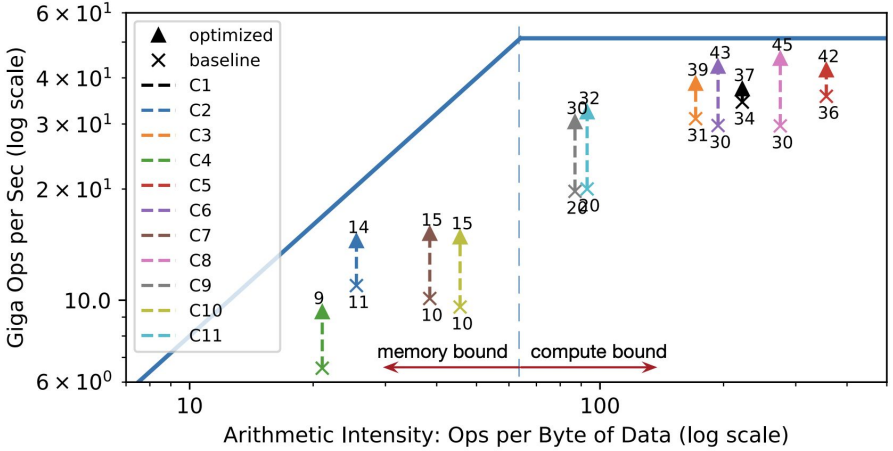
Feature Extraction

e.g. touched memory size

	xi	yi	k	xo	yo
C	2	4	4	16	64
A	1	2	16	16	64
B	2	2	16	64	64

XGBoost

cost prediction

Appendix: Memory Latency Hiding Results

- Performance improvement on all ResNet layers
- Peak compute utilization increased from 70% to 88%